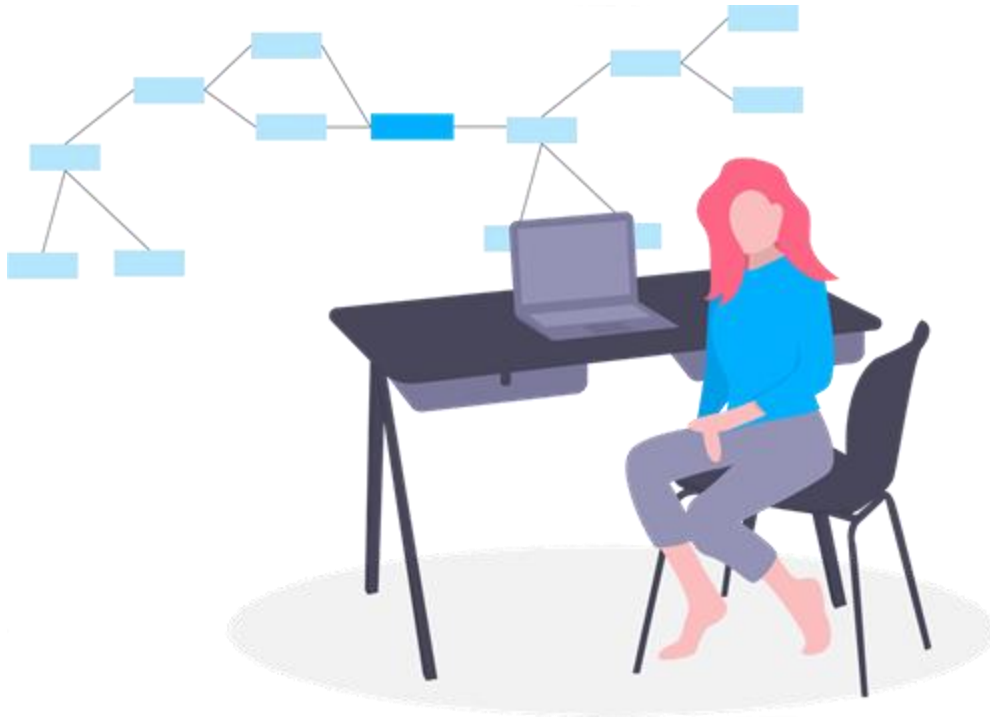




Funciones

Ing. Luis Eduardo Bello

Ing. Jose Roberto Quevedo

Funciones

Una función es un conjunto de declaraciones que toman entradas, hacen algunos cálculos específicos y producen salidas. Python proporciona funciones integradas como `print()`, `len()`, `int()`, etc., pero también podemos crear nuestras propias funciones.



```
def volumen_cilindro(altura, radio):  
    pi = 3.14159  
    return altura * pi * radio ** 2  
  
print(volumen_cilindro(10, 3))
```

• Funciones

Una función es considerada como un subprograma, ya que esta ejecuta una serie de instrucciones, siempre es buena práctica abstraer la lógica de tareas que se ejecutan múltiples veces en un programa en una función, así como que las funciones solamente haga una tarea, esto facilita la detección de errores y su corrección

Funciones

- El encabezado de la función siempre comienza con la palabra clave **def**, que indica que esta es una definición de función.
- Luego viene el nombre de la función, que sigue las mismas convenciones de nomenclatura que las variables.
- Inmediatamente después del nombre aparecen paréntesis que pueden incluir argumentos separados por comas.
- Los argumentos son valores que se pasan como entradas cuando se llama a la función y se usan en el cuerpo de la función. Si una función no toma argumentos, estos paréntesis se dejan vacíos.
- El encabezado siempre termina con dos puntos ' : '

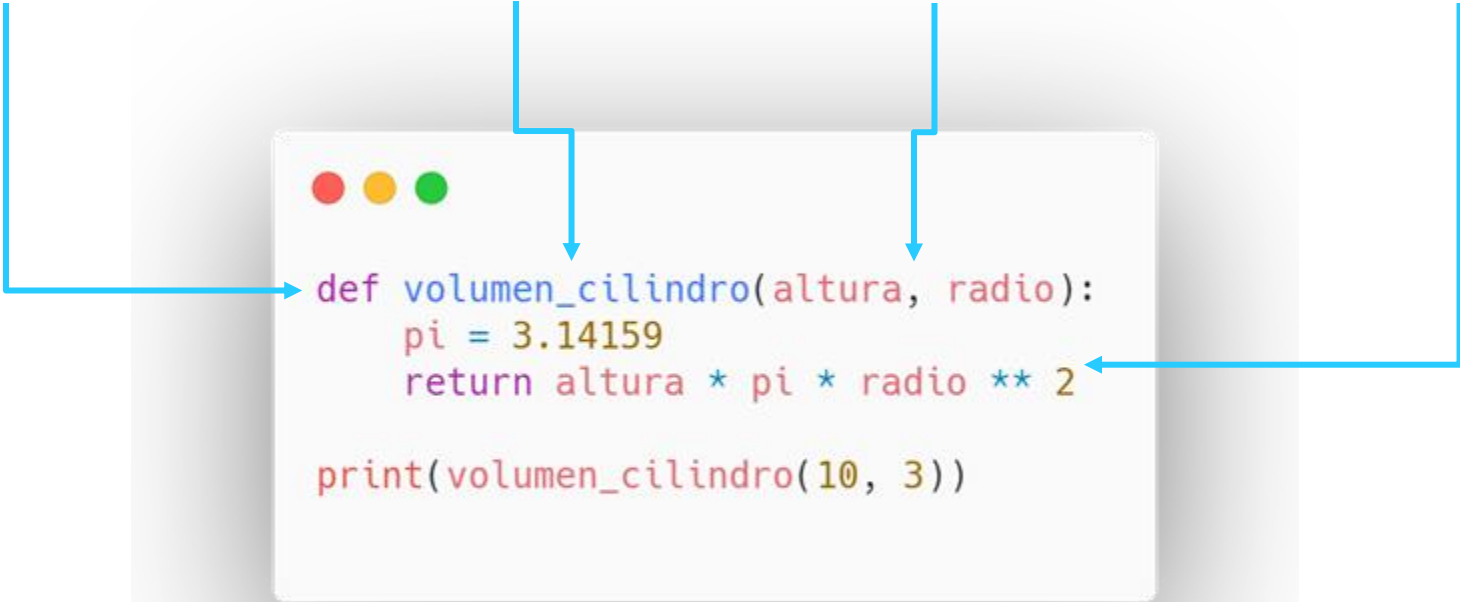
Funciones

Keyword **def**

Nombre de la Función

Parámetros

Cuerpo de la Función



```
def volumen_cilindro(altura, radio):  
    pi = 3.14159  
    return altura * pi * radio ** 2  
  
print(volumen_cilindro(10, 3))
```

The diagram illustrates the structure of a Python function definition. Four labels are positioned above a code block, with blue arrows pointing from each label to a specific part of the code. The 'Keyword def' label points to the 'def' keyword. The 'Nombre de la Función' label points to the function name 'volumen_cilindro'. The 'Parámetros' label points to the parameters 'altura, radio' in parentheses. The 'Cuerpo de la Función' label points to the function body, which includes the calculation of pi, the return statement, and the final print statement.

Cuerpo de la Función

- El cuerpo de una función es el código indentado después de la línea de encabezado.
- Dentro de este cuerpo, podemos referirnos a los argumentos y definir nuevas variables, que solo pueden usarse dentro de estas líneas indentada.
- El cuerpo puede incluir una instrucción de retorno, que consta de la palabra clave `return` seguida de una expresión que se evalúa para obtener el valor de salida para la función.
- Si no hay una declaración de `return`, la función simplemente devuelve `None`.

Cuerpo de la Función

Nueva variable

Cuerpo de la Función

Instrucción return

```
def volumen_cilindro(altura, radio):  
    pi = 3.14159  
    return altura * pi * radio ** 2  
  
print(volumen_cilindro(10, 3))
```

Referencia
Parámetros

● Parámetros

- Los parámetros deben pasarse en el mismo orden que fueron declarados.
- Existen dos tipos de parámetros los *positional*, y los *labeled*, son utilizados para cuándo un valor puede ser omitido al llamar a la función, existen en varios lenguajes de programación mientras que en otros no están permitidos.

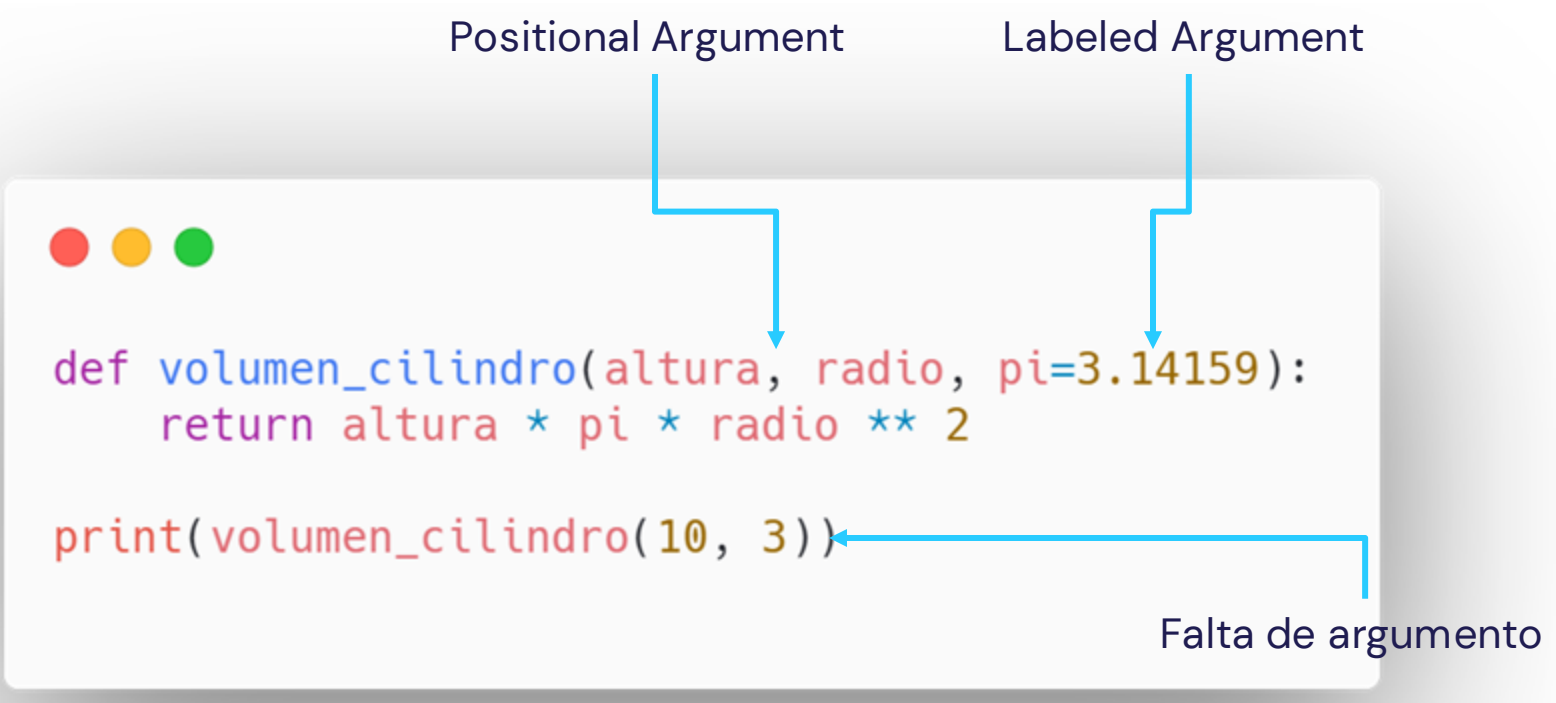
● Parámetros

- Todas las funciones en Python tienen dos formas adicionales para parámetros que el intérprete añade estos son `*args` and `**kwargs`
- `*args`: Permite que la función reciba múltiples parámetros y los guarda en una tupla
- `**kwargs`: Permite que la función reciba múltiples parámetros `labeled` y los guarda en un diccionario

Parámetros

Positional Argument

Labeled Argument



```
def volumen_cilindro(altura, radio, pi=3.14159):  
    return altura * pi * radio ** 2
```

```
print(volumen_cilindro(10, 3))
```

Falta de argumento

Parámetros



```
def volumen_cilindro(altura, radio, pi=3.14159):  
    return altura * pi * radio ** 2
```

```
print(volumen_cilindro(10, 3, 3))
```

```
print(volumen_cilindro(10, 3, pi=3))
```

```
print(volumen_cilindro(pi=3, 10, 3,)) # Error!!!
```

Equivalente

Parámetros



```
def area_rectangle(base=1, height=1):  
    return base * height
```

```
print(area_rectangle(1, 2))  
print(area_rectangle(base=1, height=2))  
print(area_rectangle(height=1, base=2))
```

Equivalente

El orden no importa
si todos son labeled

Nombre de Funciones

Los nombres de las funciones siguen las mismas convenciones de nombres que las variables.

- Solo use letras, números y guiones bajos comunes en los nombres de sus funciones.
- No pueden tener espacios y deben comenzar con una letra o un guión bajo.
- No puede usar palabras reservadas o identificadores integrados que tengan propósitos importantes en Python.
- Trate de usar nombres descriptivos que puedan ayudar a los lectores a comprender lo que hace la función.

“void”

El término *void* está asociado a lenguaje de programación fuertemente tipados, como *c/c++/java*, este *keyword* se utiliza para declarar que una función no regresa ningún valor, en Python al no tener que declarar tipos no existe este término, sin embargo, es una buena forma para referirse a las funciones que no tiene un cláusula de **return**

Return

Por otra parte las funciones que retornan un valor, se les conocen como funciones “return”, las cuales a diferencia de las “void” poseen una cláusula return que devuelve uno o varios valores después de su ejecución, en su mayoría las funciones deberían ser de este tipo, recibir parámetros y devolver un resultado.

“void” vs “return”



```
def plus_one(num):  
    print(num + 1)  
  
result = plus_one(1)  
print(type(result))
```

```
>>> <class 'NoneType'>
```



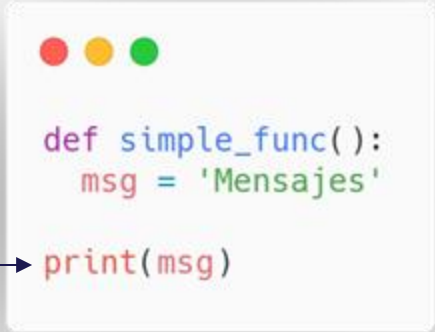
```
def plus_one(num):  
    return num + 1  
  
result = plus_one(1)  
print(type(result))
```

```
>>> <class 'int'>
```


Alcance (Scope) de las Variables

El alcance de la variable se refiere a qué partes de un programa se puede hacer referencia o usar una variable. Es importante tener en cuenta el alcance cuando se usan variables en funciones. Si se crea una variable dentro de una función, solo se puede usar dentro de esa función. No es posible acceder a él fuera de esa función.

NameError: name 'msg'
is not defined



```
def simple_func():  
    msg = 'Mensajes'  
  
print(msg)
```

The image shows a code editor window with a white background and three colored window control buttons (red, yellow, green) at the top left. It contains Python code. The first part is a function definition: `def simple_func():` followed by an indented line `msg = 'Mensajes'`. Below this, outside the function's indentation, is the line `print(msg)`. A blue arrow points from the error text below to the `print(msg)` line.

Alcance (Scope) de las Variables



```
msg = 'Mensaje'
def simple_func():
    msg = 'Mensajes'

print(msg)
```

En este caso la variable msg que está dentro de la función es única para la función o modifica a la variable msg que está afuera

```
>>> Mensaje
```

Alcance (Scope) de las Variables



```
contador = 0
```

```
def sumador():  
    contador += 12
```

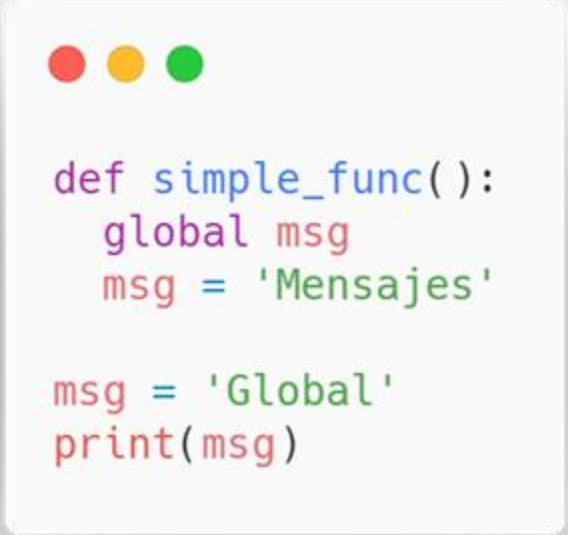
```
sumador()  
print(contador)
```

En este estamos intentado utilizar una variable local que no se encuentra declarada previamente, y como ya vimos la variable a fuera de la función no afecta a la de adentro.

```
>>> UnboundLocalError: local  
variable 'contador' referenced  
before assignment
```

Variables Globales

Una variable global, es una variable cuyo alcance (scope) es todo el programa, aunque no están prohibidas, siempre es una buena práctica evitarlas ya que hacen nuestros programas propensos a errores. Sin embargo, si es necesario utilizar una variable global deberán declararla con el keyword **global**

A code block with a light gray background and rounded corners. At the top left, there are three colored circles: red, yellow, and green. The code is written in a monospaced font with syntax highlighting: 'def' is blue, 'simple_func' is blue, '()' is black, 'global' is purple, 'msg' is red, 'Mensajes' is green, 'Global' is green, and 'print' is red.

```
def simple_func():  
    global msg  
    msg = 'Mensajes'  
  
msg = 'Global'  
print(msg)
```



DINÁMICA

Pregunta 1 - 30 puntos

Haga una función que dado un entero positivo n , encuentre la suma de todos los enteros en el rango $[1, n]$ incluidos que son divisibles por 3, 5 o 7. Devuelva un entero que denota la suma de todos los números en el rango dado que satisfacen la restricción.

Entrada: $n = 7$

Salida: 21

Explicación: Los números en el rango $[1, 7]$ que son divisibles por 3, 5 o 7 son 3, 5, 6, 7. La suma de estos números es 21.

Pregunta 2 - 30 puntos

Dado un entero n , realice una función que devuelve cualquier arreglo que contenga n enteros únicos de forma que su suma sea 0.

Entrada: $n = 5$

Salida: $[-7, -1, 1, 3, 4]$

Explicación: Estos arreglos también son aceptados $[-5, -1, 1, 2, 3]$, $[-3, -1, 2, -2, 4]$.

Pregunta 2 - 30 puntos

Haga una función que dada una lista `nums` que consta de $2n$ elementos en la forma `[x1,x2,...,xn,y1,y2,...,yn]`. Devuelve la matriz en la forma `[x1,y1,x2,y2,...,xn,yn]`.

Entrada: `nums = [2,5,1,3,4,7]`, `n = 3`

Salida: `[2,3,5,4,1,7]`

Explicación: Como `x1=2`, `x2=5`, `x3=1`, `y1=3`, `y2=4`, `y3=7`, entonces la respuesta es `[2,3,5,4,1,7]`.

Gracias!

Ing. Luis Eduardo Bello
lbello@unimet.edu.ve

Ing. Jose Roberto Quevedo
jquevedo@unimet.edu.ve

CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, and infographics & images by **Freepik**.

Please keep this slide for attribution.

